

ユニット 1・第1週

オペレーティングシステム入門

📅 第1週・日程は未定 </> 演習 TD0（解答 + 生成AIチャレンジ） ~ アニメーション 4 本

演習 TD0・解答付き

システムプログラミング環境の準備

OS を相手にプログラミングする前に、4 つのものが 필요합니다：マニュアルの読み方、プログラムのひな形、小さなユーティリティライブラリ、そして警告をエラーとして扱ってすべてをビルドする

Makefile。

</> 解答付き演習の使い方

どの演習にも完全な解答が付いています。ここに示すコードはコースの `tds/` フォルダのものそのまま、`-Wall -Wextra -Werror` でビルドし、`tds/tests/run.sh` でテストしています。出力は実際の実行結果です。あなたの作業：ビルドし、実行し、どこかを変え、何が起るかを予想して確かめること。コンパイルは常に `gcc` で、警告をエラーとして扱います。各プログラムのコマンドラインは次のとおりです（`Makefile` もまったく同じものを使って一括でビルドします）。

Bash

```
cd tds/td0
gcc -std=gnu11 -O2 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -I../common
errors.c ../common/utils.c -o errors -lm
./errors

make -C tds                # the same gcc line for every program of every lab
(make prints each one)
tds/tests/run.sh td0       # tests one lab and prepares its working folder
tds/tests/tmp/td0
```

オプション：教室内の端末

このページの中で**自分のマシンの本物の端末**を使って演習に取り組みます：再現できる記録済みの出力にはそれぞれ ▶ **実行**ボタンがあり、同じコマンドをあなたのマシンで実行します。また ▶ **端末**ボタン（演習ページの右下）を押すと、自由に入力できる端末が開きます。ブラウザは自分ではプロセスを起動できないため、コースの小さなプログラムが橋渡し（ブリッジ）をします。

Bash

```
make -C tds && tds/tests/run.sh # once: builds every lab and prepares
tds/tests/tmp/td0 ... td8
python3 tds/terminal.py # leave this terminal open while you work
```

ブリッジは**リンク**（開くと教室が自動で接続します）と、`8765.1a2b...` のような**キー**を表示します。キーはパネルに貼り付けることもできます。キーは起動するたびに変わります。**Ctrl+C** でブリッジとそのすべての端末が閉じます。

- **常に gcc で：▶ 実行**は、1行に1コマンドずつ、演習フォルダへの `cd`、コマンド内の各プログラムに対する `gcc`（実行ファイルは作業フォルダ `tds/tests/tmp/td3` に置かれます）、そして記録済み出力のコマンドを入力します。コマンドはそこで、テスト用ファイルの隣で実行されます。コードを変えてもう一度押せば効果がわかります。`gcc` がエラーを出したら何も実行されません。各演習のコードにある **gcc でコンパイル**はコンパイルだけを行います（`-D...`のすべてのバリエーション付き）。演習にまだ作業フォルダがなければ、先に `tests/run.sh` で準備します。**TDn を準備**はそれだけを行います。自分の出力を記録済みのものと比べましょう：PID、アドレス、時間は変わります。その理由を説明することも演習の一部です。
- **フローティングウィンドウ**：各端末はページの上に浮かんでいるので、課題文やマニュアル、アニメーションを読みながら使えます。バーをドラッグして（またはバーにフォーカスがあるときは矢印キーで）移動し、角でサイズを変え、-で最小化できます。置いた場所は記憶されます。+でもう1つ開けます（最大3つ）。サーバーとそのクライアント（TD2、TD7）に便利です。▶ **実行**は最後に使った端末に入力します。⌘で別ウィンドウに移すと、演習のPDFのような他のアプリよりも常に手前に表示されます（最近のFirefoxとChrome）。閉じるとページに戻ります。
- **Ubuntu**：端末（Ctrl+Alt+T）でコースのフォルダに移動して起動します。Ubuntu 付属のFirefoxでもChromeでも動きます。**Windows**：演習をビルドするWSL 2（Ubuntuの端末）の中で起動します。WSLは既定で `localhost` を転送するため、WindowsのブラウザからWSLの `127.0.0.1` に届きます。
- **セキュリティ**：ブラウザから届くシェルは扱いに注意が必要です。そのためブリッジは `127.0.0.1` でのみ待ち受け（ネットワークには決して公開しません）、そのセッションのキーを要求し、他サイトのページを拒否し、端末は最大3つまで、タブを閉じると各端末を閉じます。これはあなたのシェルで、あなたの権限で動きます：理解していないコマンドは貼り付けしないでください。

ブリッジがなくても何も変わりません：コマンドを自分の端末にコピーしてください。別のサーバー（TD7）や `agentd` デーモン（TD8）が必要な出力にはボタンがありません。手順は課題文にあります。

演習問題

- | | | | |
|---|---|---|--------------------------|
| 1 | マニュアルのページとセクション | 4 | <code>make</code> でビルドする |
| 2 | プログラムのひな形 | 5 | 自分の OS を探る |
| 3 | <code>libutils</code> ライブラリと <code>exit_if</code> | | |

TD0 ・ 1

● 解答済み

マニュアルのページとセクション

課題。このコースでは、`cmd` がマニュアルのセクション k に記載されていることを `cmd(k)` と書きます。`man -k` でキーワード検索して答えてください：C ライブラリの `printf` はどのセクションにありますか？ほかにどんなページが表示されますか？セクション `3` と `3p` の違いは何ですか？

解答

Bash

```
man -k printf | grep ^printf      # keyword search, keep the names that start
with printf
man 3 printf                      # the C library function
man 1 printf                      # the shell command with the same name
man 3p printf                    # the POSIX standard text (package
manpages-posix-dev)
```

セクション	目次	例
1	プログラムとシェルのコマンド	<code>ls(1)</code> , <code>printf(1)</code>
2	システムコール（カーネルへの入口）	<code>write(2)</code> , <code>fork(2)</code>
3	ライブラリ関数（Linux では glibc）	<code>printf(3)</code> , <code>perror(3)</code>
3p	POSIX 標準での同じ関数：どの Unix でも成り立つべきこと	<code>printf(3p)</code>
5・7	ファイル形式・概要	<code>passwd(5)</code> , <code>signal(7)</code>

答え。C の `printf` はセクション 3 にあり、`printf(1)` はシェルのコマンドです。セクション 3 は手元の実装（拡張を含む glibc）を、3p は移植可能な取り決めに説明しています。複数のシステム向けに書くときは 3p を信頼し、Linux でデバッグするときは 3 を読みましょう。

TD0 ・ 2

● 解答済み

プログラムのひな形

課題。どの演習も、いつものヘッダと `utils.h` をインクルードした空のプログラムから始めます。演習ごとに新しい名前でもコピーして始めてください。

解答



C tds/td0/skeleton.c

```
/*
 * TD0 - skeleton.c: the empty program every exercise starts from.
 * Copy it with a new name (cp td0/skeleton.c td1/echo.c) and build with
 * `make`.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    (void)argc;    /* silences -Wunused-parameter until you use them */
    (void)argv;
    return EXIT_SUCCESS;
}
```

(void)argc; の行があるのは、未使用の引数を警告する `-Wextra` と、その警告をエラーにする `-Werror` でビルドするためです。プログラムが `argc` と `argv` を使うようになったら削除してください。

TD0 ・ 3

● 解答済み・テスト済み

libutils ライブラリと exit_if

課題。utils.h を読んでください。exit_if は何をし、なぜ errno を使うのでしょうか？
ファイルを開き、存在しない場合は exit_if でエラーを報告するプログラムを書いてください。

解答

C tds/common/utils.h

```
/*
 * utils.h - small helper library shared by every lab (libutils.a).
 * Fundamentals of Operating Systems - Universidad Tecnológica Indoamerica.
 * Based on the PG204 labs (ENSEIRB-MatMeca, F. Morandat / M. Faverge).
 */
```

```

#ifndef FSO_UTILS_H
#define FSO_UTILS_H

#include <signal.h>    /* sigset_t */
#include <stddef.h>    /* size_t */
#include <sys/types.h> /* ssize_t, pid_t */

/*
 * Exit with status 1 if `condition` is true.
 * The message is `prefix: <strerror(errno)>` when errno is set, `prefix`
 otherwise.
 * errno is only meaningful right after a failed call, so use it right after
 the call you test.
 */
void exit_if(int condition, const char *prefix);

/* Write exactly `len` bytes (loops over partial writes). Returns len, or -1
 on error. */
ssize_t write_all(int fd, const void *buf, size_t len);

/* Copy everything from fd_in to fd_out until end of file (read/write loop).
 */
void transfer(int fd_in, int fd_out);

/* Naive recursive Fibonacci: a way to burn CPU time (fibo(40) takes about a
 second). */
unsigned long long fibo(int n);

/*
 * Split argv at the first "--": what comes before goes to args1, what comes
 after to args2.
 * Both arrays end with NULL. Returns -1 if a side is empty or there is no "
 --".
 */
int split_args(char *argv[], char *args1[], char *args2[]);

/* Fork a child that sleeps `seconds` seconds and exits with status 0.
 Returns its pid. */
pid_t create_waiting_child(int seconds);

/* Name of a standard signal ("SIGUSR1"), using the numbering of the running
 system. */
const char *signame(int sig);

/* Print the standard signals that belong to `set` on fd (uses write, async-

```

```

    signal-safe). */
void print_sigset(int fd, const sigset_t *set);

/* Print the pending and the blocked signals of the calling process on fd. */
void print_signals(int fd);

/* TD1, exercise 8: the structure written and read in binary by
write_struct/read_struct. */
struct nopad {
    char c1;
    long l;
    char c2;
};

#endif /* FSO_UTILS_H */

```

C tds/td0/errors.c

```

/*
 * TD0 - errors.c: how exit_if() reports a failed system call.
 * open() fails, sets errno, and exit_if prints "open missing.txt: No such
 * file or directory".
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "missing.txt";
    int fd = open(path, O_RDONLY);

    exit_if(fd == -1, path);          /* prefix = the file name */
    printf("%s opened as file descriptor %d\n", path, fd);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}

```

実際の出力

```
$ ./errors
missing.txt: No such file or directory
[exit status 1]
```

実際の出力

```
$ ./errors notes.txt
notes.txt opened as file descriptor 3
```

システムコールが失敗すると `-1` を返し、エラーコード（`ENOENT`、`EACCES` ...）をグローバル変数 `errno` に格納します。`perror` はそのコードを文字列に変えます。だから `exit_if` は確認対象の呼び出しの直後に呼ぶ必要があります。間に別の呼び出しが入ると `errno` が変わるかもしれません。

⚠️ 引き継いだ版からの修正点

シグナル名の表は macOS の番号付けを使っていました（Linux では `signame(SIGUSR1)` が別の名前を返していました）。`fibo` は `utils.h` と `utils.c` で型が異なり、`create_waiting_child` は引数を見落とし、書き込みは結果を確認していませんでした。現在、表はシステムのマクロで作られ、`write_all` 関数が部分書き込みを繰り返します。

TD0 ・ 4

● 解答済み

make でビルドする

課題。コースの `Makefile` の主なルールを説明してください：`libutils.a` がどうビルドされるか、各演習がどうコンパイルされるか、`-D` マクロで同じプログラムの複数バージョンをどう得るか。

解答



テキスト tds/Makefile

```
# The course compiles with gcc. make's built-in default is CC=cc, which "CC
?= gcc" would not change.
ifeq ($(origin CC),default)
CC := gcc
endif
CFLAGS  ?= -O2 -g
CFLAGS += -std=gnu11 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -Icommon
LDFLAGS += -pthread
LDLIBS  += -Lbin -lutils -lm

TDS := td0 td1 td2 td3 td4 td5 td6 td7
SRCS := $(foreach t,$(TDS),$(wildcard $(t)/*.c))
BINS := $(patsubst %.c,bin/%,$(SRCS))

# Variants the lab asks to build with different macros (-D...).
VARIANTS := \
    bin/td3/buffer_1 bin/td3/buffer_2 bin/td3/buffer_3 bin/td3/buffer_4
bin/td3/buffer_5 \
    bin/td3/fork_pid_0 bin/td3/fork_pid_1 bin/td3/fork_pid_2 bin/td3/fork_pid_3
\
    bin/td4/mypipe_bug \
    bin/td5/map_none bin/td5/map_read bin/td5/map_write bin/td5/map_exec
bin/td5/map_ew \
    bin/td6/philosophers_deadlock bin/td5/shm_race bin/td6/peterson_fence

.PHONY: all test clean $(TDS) td8

all: bin/libutils.a $(BINS) $(VARIANTS) td8

$(TDS): bin/libutils.a
    @$(MAKE) --no-print-directory $(filter bin/$@/%,$(BINS) $(VARIANTS))

bin/libutils.a: common/utils.c common/utils.h
    @mkdir -p bin
    $(CC) $(CFLAGS) -c common/utils.c -o bin/utils.o
    $(AR) rcs $@ bin/utils.o
```

- `bin/libutils.a` : `utils.c` をオブジェクトにコンパイルし、`ar` でアーカイブします。`.a` ファイルは静的ライブラリで、リンクは各実行ファイルに使う関数だけをコピーします。
- `bin/%: %.c` はパターンルールです: `bin/td1/echo` は `td1/echo.c` からビルドされます。`$<` は最初的前提条件、`$@` はターゲットです。

- `-Lbin -lutils` はリンクにライブラリの探し場所と使うライブラリを伝え、`-lcommon` はコンパイラに `utils.h` の場所を伝えます。
- `-D_GNU_SOURCE` は `glibc` のすべての関数（POSIX と Linux 拡張）を有効にします。`-pthread` はスレッド（TD6）で必要になります。

TD0 ・ 5

● 解答済み・実際の出力

自分の OS を探る

課題。コマンドで調べてください：使っているカーネルとディストリビューション、最初のプロセス（PID 1）、読み込まれているカーネルモジュールの数、見えているプロセッサの数、起動の各段階にかかった時間。

解答（コースのマシンでの実際の出力）

実際の出力

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod |
tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) +
16.571s (userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

- `uname` はカーネル（Linux 7.0）を、`/etc/os-release` はディストリビューション（Ubuntu 26.04）を表示します。1つのカーネルが多くのディストリビューションで使われています。
- PID 1 は `systemd` で、カーネルが起動時に最初に起動するプロセスです。他のすべてのプロセスはこれの子孫です。
- 読み込まれたモジュールは 258 個：Linux はモノリシックですが、ドライバは動的に読み込み・解放されます。
- `systemd-analyze` は起動をファームウェア、ローダ、カーネル、ユーザー空間に分けます：カーネルが最も短い段階です。（WSL やコンテナでは使えないことがあります。）

これらの時間はアニメーション A1.3・起動の流れで使われます →

チェックリスト

生成AIチャレンジ・ テスト付きの自分の Makefile



目標。 AI の助けを借りて、演習の各問題をそれぞれ独立したバイナリにビルドし、既知の入力で各プログラムを実行する `test` ターゲットを加えた Makefile を書いてください。

最初のプロンプト。 好きなツール (Claude、ChatGPT、Gemini、Copilot、Codex...) で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

C によるシステムプログラミングの家庭教師として振る舞ってください。skeleton.c、utils.c、utils.h (ライブラリ libutils.a) が入ったフォルダがあります。次の Makefile がほしいです：(1) libutils.a を一度だけビルドする、(2) フォルダ内の各 .c を -Wall -Wextra -Werror で実行ファイルにビルドする、(3) 各プログラムを実行して出力を .expected ファイルと比較する `test` ターゲットを持つ。まず使うパターンルール (%) と自動変数 (\$@、\$<、\$^) を説明し、次に 1 行ずつコメントを付けた Makefile を示してください。

結果の確かめ方

- `make` を 2 回実行する：2 回目は何も再ビルドされない。
- utils.h に触れる (`touch common/utils.h`) と、それに依存するものがすべて再ビルドされる。
- 期待される出力をわざと 1 つ壊す：`make test` は失敗しなければならない。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。